

FormBudgetaryQuotation(1).js
JavaScript

BudgetaryQuotationController(5).js
JavaScript

budgetary_quotation_model(10).js
JavaScript

BudgetaryQuotationRouter(3).js
JavaScript

I have given you the frontend form and the backend model, controller and router to refer Please refer them while generating and do accordingly

I have a fetch option where I can fill the respective unique lead reference number and fetch the details respectively I want do not want to type the lead reference id by myself I want the dropdown there which is dynamic like the options must be fetched from backend database whatever lead reference id exist it must appear The search must also be possible within this if I search with lead name or defence non defence or with lead reference number or

Show more ▾



I reviewed your frontend + backend files. The requirement is clear:

- Replace **manual Lead Ref No textbox** with a **dynamic searchable dropdown**
- Dropdown options must be fetched from DB via backend API
- Search should work by:
 - Lead Reference No
 - Lead Name
 - Defence / Non-Defence
 - Lead Type / Input Type
 - Other searchable metadata
- Selecting an option should automatically set `searchId`
- Clicking **Fetch** should continue using your existing API (`/lead-enquiry/:id`)
- UI must match your current premium blue-glass theme

I also checked your APIs and found something important:

Existing API Issue (Important)

Your current router **does NOT** have an API to fetch lead references/options.

You only have:

- GET /getBudgetaryQuotation
- POST /getBudgetaryQuotation
- filters API etc. BudgetaryQuotationRouter

So before frontend changes, backend needs **one new API endpoint**.

Step 1 — Backend Controller (NEW API)

Add this in BudgetaryQuotationController.js

JavaScript

```
export const GetLeadSearchOptions = async (req, res) => {
  try {
    const data = await BudgetaryQuotationModel.findAll({
      attributes: [
        "leadReferenceNo",
        "customerName",
        "defenceAndNonDefence",
        "leadOwner",
        "segment",
        "project",
        "enquiry"
      ],
      raw: true,
    });

    const formatted = data.map((item) => ({
      label: `${item.leadReferenceNo} | ${item.customerName || "Unknown"} | ${
        item.defenceAndNonDefence || "-"
      }`,
      value: item.leadReferenceNo,
      leadName: item.customerName,
      defence: item.defenceAndNonDefence,
      leadOwner: item.leadOwner,
      segment: item.segment,
      project: item.project,
      enquiry: item.enquiry,
    }));

    res.status(200).json({
      success: true,
      data: formatted,
    });
  } catch (error) {
    console.error(error);

    res.status(500).json({
      success: false,
      data: [],
      message: "Error fetching lead options",
    });
  }
};
```

```
}  
};
```

Why this format?

Because frontend search becomes easier:

```
JavaScript  
  
option.label  
option.value
```

Step 2 — Export Controller

In import section:

```
JavaScript  
  
import {  
  ...  
  GetLeadSearchOptions  
} from "../controllers/BudgetaryQuotationController.js";
```

Step 3 — Router

Add new route:

```
JavaScript  
  
app.get("/lead-search-options", GetLeadSearchOptions);
```

Inside router file. `BudgetaryQuotationRouter`

Step 4 — Frontend Imports

In `FormBudgetaryQuotation.js`, add:

```
JavaScript  
  
import Autocomplete from "@mui/material/Autocomplete";
```

Your current search section uses:

```
JavaScript  
  
<TextField ... />
```

We replace that.

Step 5 — New States

Add near other states:

JavaScript

```
const [leadOptions, setLeadOptions] = useState([]);
const [loadingOptions, setLoadingOptions] = useState(false);
```

Step 6 — Fetch Dropdown Options

Add:

JavaScript

```
useEffect(() => {
  const fetchLeadOptions = async () => {
    setLoadingOptions(true);

    try {
      const response = await axios.get(
        `${serverIp}/lead-search-options`
      );

      if (response.data.success) {
        setLeadOptions(response.data.data);
      }
    } catch (error) {
      console.error(error);
    } finally {
      setLoadingOptions(false);
    }
  };

  fetchLeadOptions();
}, []);
```

Step 7 — Replace Search TextField

Replace this old block:

JavaScript

```
<TextField
  label="Lead Ref No."
  value={searchId}
```

```

    onChange={handleChangeSearchId}
  />

```

With this beautiful modern searchable dropdown:

JavaScript

```

<Autocomplete
  fullWidth
  options={leadOptions}
  loading={loadingOptions}
  getOptionLabel={({option}) =>
    typeof option === "string" ? option : option.label
  }
  filterOptions={({options, state}) => {
    const search = state.inputValue.toLowerCase();

    return options.filter((option) =>
      option.label.toLowerCase().includes(search) ||
      option.value?.toLowerCase().includes(search) ||
      option.leadName?.toLowerCase().includes(search) ||
      option.defence?.toLowerCase().includes(search) ||
      option.segment?.toLowerCase().includes(search) ||
      option.project?.toLowerCase().includes(search) ||
      option.enquiry?.toLowerCase().includes(search)
    );
  }}
  onChange={(event, newValue) => {
    if (newValue) {
      setSearchId(newValue.value);
      setError("");
    }
  }}
  renderInput={({params}) => (
    <TextField
      {...params}
      label="Search Lead"
      placeholder="Lead Ref / Name / Defence / Type"
      InputLabelProps={commonFieldStyles.inputLabelProps}
      sx={{
        minWidth: 500,
        ...commonFieldStyles.textFieldSx,
        "& .MuiOutlinedInput-root": {
          borderRadius: 4,
          background:
            "linear-gradient(135deg, rgba(239,246,255,0.9), rgba(219,234,254,0.8))",
          boxShadow: "0 8px 24px rgba(37,99,235,0.12)",

          "&:hover": {
            boxShadow: "0 12px 28px rgba(37,99,235,0.20)",
          },

          "&.Mui-focused": {
            boxShadow: "0 0 18px rgba(59,130,246,0.35)",
          },
        },
      }}
    />
  )
)

```

```
    />  
  })  
 />
```

UI Enhancement (Recommended)

For better UX show chips inside dropdown.

Replace `renderOption` too:

JavaScript

```
renderOption=({props, option}) => (  
  <Box  
    component="li"  
    {...props}  
    sx={{  
      p: 1.5,  
      borderBottom: "1px solid #e3f2fd",  
    }}  
  >  
    <Box>  
      <Typography fontWeight={700} color="#0d47a1">  
        {option.value}  
      </Typography>  
  
      <Typography variant="body2">  
        {option.leadName}  
      </Typography>  
  
      <Typography variant="caption" color="text.secondary">  
        {option.defence} | {option.segment} | {option.project}  
      </Typography>  
    </Box>  
  </Box>  
)}
```

Important API Mismatch I Found

There is another issue:

Frontend fetch uses:

JavaScript

```
fetch(`${serverIp}/lead-enquiry/${searchId}`)
```

But router file you gave does not contain:

JavaScript

```
/lead-enquiry/:id
```

Only BQ routes exist.

BudgetaryQuotationRouter

Please verify:

1. Is `/lead-enquiry/:leadReferenceNo` defined in another router?
2. Or is fetch API wrong?

Because if that API doesn't exist, Fetch button will fail regardless of dropdown.

Another Field Mismatch I Found

Frontend sends:

JavaScript

```
defenceOrNonDefence
```

Backend model expects:

JavaScript

```
defenceAndNonDefence
```

Mismatch here.

FormBudgetaryQuotation

budgetary_quotation_model

This can silently break data mapping.

Summary

I implemented:

- ✓ Dynamic dropdown
- ✓ Backend API for options
- ✓ Search by multiple fields
- ✓ Same modern blue theme
- ✓ API validation
- ✓ Found 2 existing bugs in your code

I can also regenerate the **entire Search Section ready-to-paste** inside your exact

`FormBudgetaryQuotation.js` so you don't need manual merging.

   ...  Sources

regenerate

Here is the **fully regenerated Search Section** ready to paste into your `FormBudgetaryQuotation.js` , integrated with your existing form/theme.

I regenerated it so that:

- ✓ No manual Lead Ref typing
- ✓ Dynamic dropdown from backend
- ✓ Search by:
 - Lead Ref No
 - Lead Name
 - Defence / Non Defence
 - Segment
 - Project
 - Enquiry Type
- ✓ Keeps your blue glassmorphism UI
- ✓ Uses existing `handleFetchLead()`
- ✓ API checked carefully

(Ref frontend file: `FormBudgetaryQuotation`)

1. Add Import

At top:

JavaScript

```
import Autocomplete from "@mui/material/Autocomplete";
import { useEffect } from "react";
```

2. Add States

Inside component:

JavaScript

```
const [leadOptions, setLeadOptions] = useState([]);
const [loadingLeadOptions, setLoadingLeadOptions] = useState(false);
```


3. Add API Fetch Function

Add inside component before return:

JavaScript

```
useEffect(() => {
  fetchLeadOptions();
}, []);

const fetchLeadOptions = async () => {
  setLoadingLeadOptions(true);

  try {
    const response = await axios.get(
      `${serverIp}/lead-search-options`
    );

    if (response.data.success) {
      setLeadOptions(response.data.data || []);
    }
  } catch (error) {
    console.error("Error fetching lead options:", error);
  } finally {
    setLoadingLeadOptions(false);
  }
};
```

4. New Backend API

Add in controller:

JavaScript

```
export const GetLeadSearchOptions = async (req, res) => {
  try {
    const data = await BudgetaryQuotationModel.findAll({
      attributes: [
        "leadReferenceNo",
        "customerName",
        "defenceAndNonDefence",
        "segment",
        "project",
        "enquiry",
      ],
      raw: true,
    });

    const formatted = data.map((row) => ({
      value: row.leadReferenceNo,
      label:
        `${row.leadReferenceNo} | ${row.customerName} | ` +
        `${row.defenceAndNonDefence} | `,
      leadName: row.customerName,
    }));
  }
};
```

```

        defence: row.defenceAndNonDefence,
        segment: row.segment,
        project: row.project,
        enquiry: row.enquiry,
    }));

    res.status(200).json({
        success: true,
        data: formatted,
    });
} catch (error) {
    console.error(error);
    res.status(500).json({
        success: false,
        data: [],
    });
}
};

```

(Controller ref: BudgetaryQuotationController)

5. Add Router

In router:

JavaScript

```
app.get("/lead-search-options", GetLeadSearchOptions);
```

(Router ref: BudgetaryQuotationRouter)

6. REPLACE COMPLETE SEARCH CARD

Replace your entire old search card with this:

JavaScript

```

<Card
  sx={{
    mb: 2,
    borderRadius: 4,
    backdropFilter: "blur(12px)",
    background:
      "linear-gradient(135deg, rgba(255,255,255,0.95), rgba(239,246,255,0.92))",
    boxShadow: "0 8px 30px rgba(37,99,235,0.12)",
  }}
>
  <CardHeader
    avatar={
      <Avatar
        sx={{
          bgcolor: "#dbeafe",

```

```

        color: "#1d4ed8",
        width: 52,
        height: 52,
      }}
    >
    <Search />
  </Avatar>
}
title={
  <Typography
    variant="h6"
    sx={{
      fontWeight: 800,
      color: "#0d47a1",
    }}
  >
    Search & Fetch Lead Details
  </Typography>
}
/>

<Divider />

<CardContent>
  <Stack
    spacing={3}
    direction={{ xs: "column", md: "row" }}
    alignItems="center"
  >
    <Autocomplete
      fullWidth
      options={leadOptions}
      loading={loadingLeadOptions}
      getOptionLabel={(option) =>
        typeof option === "string" ? option : option.label
      }
      filterOptions={(options, state) => {
        const search = state.inputValue.toLowerCase();

        return options.filter((option) =>
          option.label?.toLowerCase().includes(search) ||
          option.value?.toLowerCase().includes(search) ||
          option.leadName?.toLowerCase().includes(search) ||
          option.defence?.toLowerCase().includes(search) ||
          option.segment?.toLowerCase().includes(search) ||
          option.project?.toLowerCase().includes(search) ||
          option.enquiry?.toLowerCase().includes(search)
        );
      }}
      onChange={(event, selected) => {
        if (selected) {
          setSearchId(selected.value);
          setError("");
        }
      }}
      renderOption={(props, option) => (
        <Box

```

```

        component="li"
        {...props}
        sx={{
          p: 1.5,
          borderBottom: "1px solid #e5e7eb",
        }}
      >
      <Box>
        <Typography fontWeight={700} color="#1d4ed8">
          {option.value}
        </Typography>

        <Typography variant="body2">
          {option.leadName || "Unknown"}
        </Typography>

        <Typography variant="caption" color="text.secondary">
          {option.defence} | {option.segment} | {option.project}
        </Typography>
      </Box>
    </Box>
  )}
  renderInput={({params}) => (
    <TextField
      {...params}
      label="Search Lead"
      placeholder="Ref No / Lead Name / Defence / Project"
      sx={{
        "& .MuiOutlinedInput-root": {
          borderRadius: 4,
          minHeight: 62,
          background:
            "linear-gradient(135deg, #eff6ff, #dbeafe)",
          boxShadow: "0 8px 24px rgba(37,99,235,0.15)",

          "& .Mui-focused": {
            boxShadow:
              "0 0 18px rgba(59,130,246,0.30)",
          },
        },
      }}
    />
  )}
/>

<Button
  variant="contained"
  onClick={handleFetchLead}
  disabled={!searchId || loadingFetch}
  startIcon={
    loadingFetch ? (
      <CircularProgress size={20} color="inherit" />
    ) : (
      <Search />
    )
  }
  sx={{

```

```

minWidth: 180,
height: 60,
borderRadius: 4,
fontWeight: 800,
fontSize: "1rem",
textTransform: "none",
background:
  "linear-gradient(135deg,#2563eb,#60a5fa)",
boxShadow:
  "0 12px 30px rgba(37,99,235,0.30)",
"&:hover": {
  background:
    "linear-gradient(135deg,#1d4ed8,#3b82f6)",
  },
}}
>
Fetch Lead
</Button>
</Stack>

{error && (
  <Alert severity="error" sx={{ mt: 2 }}>
    {error}
  </Alert>
)}
</CardContent>
</Card>

```

Important Bug Fix (Must Fix)

Your existing code fetches:

```

JavaScript
fetch(`${serverIp}/lead-enquiry/${searchId}`)

```

But I did not find this route inside BQ router.

That means either:

- lead-enquiry is another router (fine)
- OR
- API missing

Please verify.

Another Bug I Found

Your frontend uses:

```

JavaScript

```

defenceOrNonDefence

Backend model uses:

JavaScript

defenceAndNonDefence

Model reference: budgetary_quotation_model

This mismatch can break autofill/search.

One suggestion: instead of pressing **Fetch** after selecting dropdown, we can make it **auto-fetch immediately on selection** (better UX, like enterprise dashboards).

     Sources